

CS455 Software Engineering — Deliverable 0: Project Proposal

Caf@IITK: Campus Cafeteria Pre-Order and Pickup-Slot Platform

Team XForce

Krishna Kumayu, 230576 | Kshitij Gupta, 230581 | Rudransh Verma, 230881 | Shashi Bhidodiya, 230956 | Suryansh Verma, 231061

29 August 2026

1. Project Title and Description

Caf@IITK — Campus Cafeteria Pre-Order and Pickup-Slot Platform. Students pre-order meals from a single campus cafeteria and reserve a seat in a limited-capacity pickup window, avoiding the lunch-hour queue. A confirmed order consumes prepared-meal inventory and one pickup-slot seat; modification and cancellation restore both under a defined policy. A rule engine computes the payable amount from item prices, time-of-day discounts and subsidy eligibility, and a Kitchen Planning Agent drafts shortage forecasts and preparation plans for administrator approval. Payments are simulated, so the engineering focus stays on transactional consistency rather than payment integration.

2. User Roles

- **Student** — browses the live menu, places an order and reserves a pickup slot, modifies items or slot before preparation begins, cancels until the cut-off, tracks order status, and views past orders with their price breakdown. Can act only on their own orders.
- **Cafeteria Administrator** — onboards and removes menu items, sets and replenishes the daily prepared-meal inventory for each item, configures the service window and pickup-slot capacities, sets pricing and discount rules, approves or rejects Kitchen Planning Agent proposals, and inspects the audit log.
- **Kitchen Staff** — views the preparation queue for the current and next slot, advances order state (Preparing → Ready → Picked Up / No-show), and flags an item temporarily unavailable. Cannot change prices or inventory, or cancel a student's order.

Onboarded / removed entity: menu items, managed by the administrator; removal is a soft-delete so that in-flight orders and historical price records stay valid.

3. Core Transactional Workflow

Cart → slot selection → price computation → order confirmation → Accepted → Preparing → Ready → Picked Up.

- **Create:** in a single database transaction the system re-validates availability, decrements the prepared-meal inventory of each item and the remaining capacity of the chosen slot, stores the price snapshot, and writes an audit entry. Mock payment authorisation follows; a decline or timeout releases both reservations. An idempotency key ensures a duplicate submission cannot create a second order.
- **Modify:** permitted only while the order is Accepted. The system computes the difference between the old and new basket and slot, releasing the old reservations and acquiring the new ones in one transaction and re-pricing the order; if the new slot or item is unavailable, the whole modification rolls back and the original order is untouched.
- **Cancel:** permitted until the cancellation cut-off. The slot seat is always released; the meal is returned to available inventory only if preparation has not started, and a mock refund is recorded exactly once.
- **Audit:** every state transition is appended to a log with actor, timestamp and before/after resource counts.

4. Shared Resource and Concurrency Challenge

- **Contended resources:** prepared-meal inventory (a fixed number of portions per item per day) and pickup-slot seats (a fixed number per 15-minute window). No portion or seat may ever be allocated twice, and neither count may go negative.
- **The challenge:** at menu release many students target the same popular item and the same early slot within seconds. A naive read-check-write produces lost updates, oversold slots and negative inventory.
- **Planned solution:** one short ACID transaction per order using pessimistic row locks (SELECT ... FOR UPDATE) acquired in a fixed canonical order to avoid deadlock, optimistic version columns on the order for modify/cancel races, database CHECK constraints as a final guard, and idempotency keys for duplicate submissions. Optimistic-only retry, SERIALIZABLE isolation and Redis-based counters will be evaluated as alternatives and the trade-offs documented.

5. AI Agent Placement, Tools and Rationale

- **Agent:** a Kitchen Planning Agent, invoked on a schedule before each service window and on demand from the admin dashboard. It summarises pending demand, forecasts which items will run out, recommends which item to temporarily disable, proposes a preparation priority across the queue, and drafts a replenishment plan for the next window.
- **Tools it may use:** read-only — `get_pending_orders`, `get_inventory_levels`, `get_slot_utilisation`, `get_historical_demand`, `get_menu_config`. Write-shaped tools produce proposals only — `propose_item_disable`, `propose_prep_priority`, `propose_replenishment_plan`.
- **Boundaries:** autonomous for analysis and drafting; administrator approval required to disable an item, reorder the queue or issue a replenishment plan; never permitted to change prices, inventory or any student order. The agent's service account holds no credential that can mutate orders or inventory, so the prohibition is structural

rather than a prompt instruction. Outputs are schema-validated and bounds-checked, and every tool call and decision is logged.

- **Why here:** the agent performs real planning work inside the operational workflow — reasoning over live inventory, demand and slot utilisation to produce decisions the kitchen would otherwise make manually — rather than being a chatbot attached to the application.

6. Proposed Technology Stack

- **Client (Flutter):** a single Flutter codebase in Dart targeting Flutter Web, with three role-scoped dashboards selected at login. Riverpod for state management, Dio for HTTP with retry and idempotency-key headers, GoRouter for role-guarded navigation, and a Dart API client generated from the OpenAPI schema FastAPI publishes automatically, so the contracts cannot drift.
- **Backend:** Python 3.12 + FastAPI, with SQLAlchemy 2.0, Pydantic v2 validation and JWT authentication with role-based dependencies. PostgreSQL 16 is the transactional store, with row-level locking expressed through SQLAlchemy's `with_for_update()`. A relational backend is retained deliberately, since the concurrency requirement depends on row-level locking and ACID transactions rather than on the application runtime.
- **AI layer:** the Anthropic/OpenAI Python SDK called from a dedicated backend agent service (never from the Flutter client, so no API key ever ships in the app), with structured tool-calling, Pydantic models as the validation schema for every tool argument and agent output, a policy gate, and a deterministic heuristic fallback.
- **Testing and quality:** Flutter — `flutter_test`, `mocktail` and `integration_test`. Backend — `pytest` with `pytest-asyncio`, `testcontainers-python` against a real PostgreSQL, `httpx` for API tests, and `pytest-cov` (≥80% gate).
- **Ops:** Docker Compose locally; GitHub Actions CI running `pytest`, `ruff`, `mypy`, `flutter analyze`, `flutter test`, the coverage gate and a Flutter Web build; backend served by `uvicorn/gunicorn` with multiple workers on AWS EC2 + RDS free tier, web client alongside; structured JSON logging with correlation IDs from the client.
- **Alternative considered:** Flutter + Firebase would be faster to build, but Firestore's transaction and contention model would weaken the concurrency demonstration central to this project, so it is recorded as a rejected option.

7. Initial Risks and Technical Uncertainties

- **Concurrency correctness under load** — lock contention or deadlock may degrade latency. Mitigation: canonical lock ordering, no external calls inside transactions, and a concurrency harness built in the first sprint so problems surface early.
- **Compensation logic for modify and cancel** — partial restores and duplicate refunds are the likeliest source of subtle defects. Mitigation: an explicit state machine with enumerated legal transitions, plus regression tests for every fixed defect.
- **Flutter learning curve and cross-platform effort** — three dashboards across mobile and web may absorb more time than expected. Mitigation: a shared widget library, Web as the primary demo target, and a UI spike in Sprint 0.
- **LLM agent unreliability** — hallucinated items, malformed tool arguments or unsafe proposals. Mitigation: strict Pydantic output schemas, a whitelisted tool registry, referential and bounds validation, mandatory human approval, and a heuristic fallback when the API fails.
- **Cost and free-tier limits** — load tests and agent runs can exhaust the AWS free tier and the API budget. Mitigation: local containerised load testing by default, a per-day call cap, cached agent responses, and budget alarms.
- **Scope creep** — real payments, multiple cafeterias or delivery would crowd out the mandatory work. Mitigation: mock payments, one cafeteria and roughly 15 menu items, with new ideas recorded in the backlog as deferred.
- **Traceability discipline** — maintaining genuine Jira-to-GitHub links all semester. Mitigation: a branch and commit naming convention enforced by a CI check for the Jira key.

8. Jira / GitHub Development Strategy and Links

- **Jira (Scrum, project key CAF):** functional and non-functional requirements recorded as issues labelled FR-n / NFR-n, organised into epics (identity, ordering, inventory, slots, rules, agent, Flutter client, testing, deployment). Work is broken into user stories, tasks and sub-tasks assigned to individual members with priority, estimates, components and issue links, and bugs are tracked as issues linked to the regression test added. Sprints are two weeks, aligned to the deliverable dates (26 Sep, 15 Oct, 6 Nov), with a dashboard showing burndown, velocity and open defects.
- **GitHub:** a repository with `/backend`, `/app` (Flutter), `/docs` and `/Deliverables`; protected main, a develop integration branch, and feature branches named CAF-`<issue>-<slug>`. Every commit message begins with the Jira key (e.g. "CAF-42: lock inventory and slot rows before decrement"). Significant changes merge via pull requests carrying a description, test evidence and the Jira link, with at least one peer review; Actions runs both builds, the tests and the coverage gate. The GitHub for Jira integration links branches, commits and PRs to their issues, giving the chain Requirement → Story → Jira issue → branch → commits → PR → test case → test result.
- Jira — Will be available soon · GitHub — <https://github.com/XForce-IITK/Caf-IITK>
- **Access:** the instructor and the assigned TAs will be added to both projects